

POST-MORTEM REPORT: INC-101

Title: Redis Cache Failover

Severity: ● High

Incident ID: INC-101

Status: Resolved

Duration: 30 minutes (2026-04-12T08:17:20Z – 2026-04-12T08:47:20Z)

1. Executive Summary

On April 12, 2026, at 08:17:20Z, the **Redis Cache** primary node experienced a health check failure, leading to a brief service disruption. **User Service** and **Payment Service** were immediately affected due to their dependency on Redis for session management and transaction state caching. The platform's automated Sentinel configuration triggered a failover to a replica, resolving the incident at 08:47:20Z.

2. Timeline

- **08:17:20Z:** Primary Redis node fails health check; alert triggered.
- **08:19:00Z:** Redis Sentinel confirms failure and initiates failover.
- **08:21:45Z:** Replica promoted to primary; services begin reconnecting.
- **08:25:00Z:** High error rates detected in **User Service** as cache warming begins.
- **08:47:20Z:** All services stabilize; error rates return to baseline.

3. Root Cause Analysis

The primary Redis node encountered a transient failure during a scheduled internal health check, likely due to an underlying infrastructure issue (possibly a networking flap or disk I/O spike). Sentinel correctly identified the failure, but the propagation of the new primary's identity across the application layer was slower than expected, leading to the 30-minute recovery period.

4. Business Impact

- **User Service:** ~15% of active sessions were terminated, forcing users to re-log.
- **Payment Service:** 22 transactions were temporarily held in a pending state before successfully re-syncing with the cache post-failover.
- **Blast Radius:** High impact on authentication and checkout flows.

5. Remediation & Prevention

- **Remediation:** Automated failover via Sentinel was the primary recovery mechanism.
- **Prevention:**
 1. **Sentinel Quorum Optimization:** Increase the number of sentinels to reduce false negatives and speed up quorum consensus.
 2. **Application-Layer Resilience:** Implement circuit breakers and retry logic with

exponential backoff in the **User** and **Payment** services.

3. **Cluster Transition Tuning:** Optimize `maxmemory-policy` and `client-output-buffer-limit` to handle rapid reconnection bursts.